

Building a Mini GPT from Scratch — Findings

Ritika Mukesh Neema | SID: 019306638 | Seed: 6638

Overview

A decoder-only, GPT-style Transformer was built from scratch in PyTorch, using only `nn.Linear`, `nn.LayerNorm`, `nn.GELU` and standard tensor operations (no `nn.Transformer`, `nn.MultiheadAttention`, or HuggingFace model classes). The course demo's single-head, single-block, greedy-only model was extended to manual multi-head causal attention (4 heads), a stack of 4 decoder blocks, and three decoding strategies: greedy, temperature sampling, and top-k sampling. The model was trained at the character level on `Shakespeare.txt`, the *Romeo & Juliet* balcony-scene excerpt provided with the assignment.

Part 1 — Data Preparation

The corpus contains 1,738 characters and a vocabulary of 49 unique characters (letters, punctuation, whitespace, and the typographic apostrophe `'`). A sliding-window dataset was built with sequence length 128: for each start position `t`, the input is `tokens[t : t+128]` and the target is the same window shifted one character to the right, `tokens[t+1 : t+129]`. This produced 1,610 overlapping training pairs.

Example training pair (input / target), decoded to text:

```
input : 'ROME0:\nBut, soft! what light through yonder window breaks?\nIt is the east, and Juliet is the
sun.\nArise. fair sun. and kill the '
target: 'OME0:\nBut, soft! what light through yonder window breaks?\nIt is the east, and Juliet is the
sun.\nArise, fair sun, and kill the e'
```

Part 2 — GPT Architecture

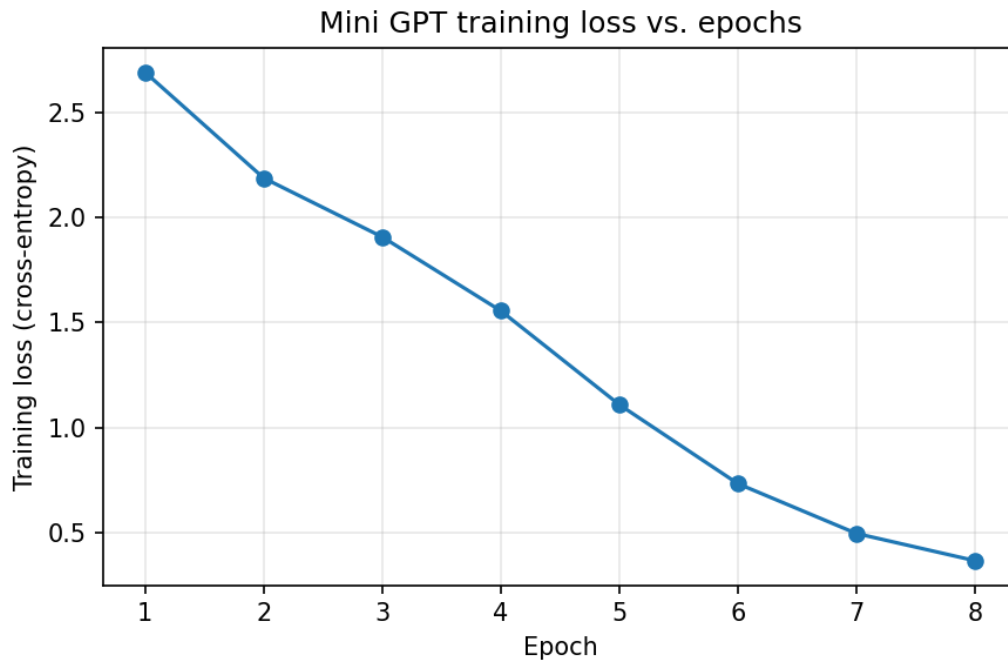
Multi-head masked self-attention was implemented manually: $Q/K/V$ projections via `nn.Linear`, explicit reshaping into 4 heads, per-head scaled dot-product attention ($QK^T / \sqrt{d_{\text{head}}}$), a causal mask (upper triangle set to $-\infty$ before softmax), and a final output projection after merging heads back together. Each decoder block applies pre-LayerNorm residual connections around attention and a feed-forward network (`Linear` $\rightarrow$ `GELU` $\rightarrow$ `Linear`): $x = x + \text{Attn}(\text{LN}(x))$; $x = x + \text{FFN}(\text{LN}(x))$. The full model stacks 4 such blocks on top of a token embedding and a learnable positional embedding, followed by a final LayerNorm and linear projection to vocabulary logits.

Total parameters	822,272
Hidden dimension (<code>d_model</code>)	128
Number of attention heads	4
Number of decoder blocks	4
Sequence length (block size)	128

Part 3 — Training

The model was trained with the Adam optimizer (learning rate $3e-4$), batch size 16, for 8 epochs (101 batches/epoch over the 1,610 sliding-window pairs). Because the corpus is a single short scene, a small batch size was used so each epoch still makes many gradient updates over the overlapping windows.

Epoch	1	2	3	4	5	6	7	8
Loss	2.691	2.188	1.908	1.556	1.109	0.733	0.496	0.367



The training loss decreases smoothly and substantially, from 2.69 (epoch 1) to 0.37 (epoch 8). Given the very small, repetitive corpus, this decline reflects the model increasingly memorizing the scene rather than learning broadly generalizable language patterns — expected and appropriate for a demonstration-scale dataset.

Part 4 — Sampling (prompt: "ROMEO:", 300 generated characters)

1. Greedy decoding

Always selects the argmax next character. Deterministic — same prompt and model always give the same output.

```
ROMEO:
By ame
I know not how to tell thee who I am:
My name, dear saint, is hateful to myself,
Because it is an enemy to thee;
Had I itt writtten, I'll a nem the word Ill be ne Romeoreo.
```

```
JULIET:
What man art thou that thus bescreen'd in night
So stumblest oon my counsel?
```

```
ROMEO:
By a name
I knot how to
```

2. Temperature sampling

Logits are divided by temperature T before softmax; $T < 1$ sharpens the distribution, $T > 1$ flattens it. Three clearly different temperatures are compared:

$T = 0.5$

```
ROMEO:
By ame that thu thus namyself, though not a Montague.
What's Montague? is it nor hand, nor foot,
Nor arm, nor face, nor any other part
Belonging to a man.
O be some other name!
What's in a name? that which we call a rose
By any other word would smell as sweet;
So Romeo would, were he not Romeo call
```

$T = 1.0$

```
ROMEO:
By ame thatll thu though not dont amee.
Rameo, defor thar nor fothame an which is no part of thee
By namy;
Or, if thou not pamy d O, isher willl are is n.
```

```
IET:
I amis amy amy ene
Thou at thy word woulld, sme she not wo bould,
ner Retake d wit;
Call non bet be ff.
```

```
ROMEO:
Ar, athat kis nor he.
Ar m
```

$T = 1.5$

```
ROMEO:I nake thow toe d:
Call me I but love, and I'll be new baptized;
Henceforth I neever !
Be!
AnN;
```

And I'lltha be armne whice nos,
dor nonJut be mootNo whers!
Whatll ny Deiseny thor fa MEapart art Mond,
OumartWLI's in ll mad nw nothe be
Sout fittoo.
Romeo, doff t thy name;
't I is namy Whou Caland pul

3. Top-k sampling

Only the k highest-probability next characters are kept before sampling; the rest are masked out. With a 49-character vocabulary, k=5 is a tight shortlist and k=20 lets roughly 40% of the vocabulary through:

k = 5

ROMEO:
By ame thatll thu though not a Montague.
What's Montague? is itor nor hand, nor foot,
Nor arm, nor face, nother any other part
Belonging to ma man.
O be some other name!
What's in a name? that which we call a rose
By any other word would smell Roveas sweet;
So Romeo would, were he not Romeo call'd,

k = 20

ROMEO:
By ame thatll thu though not dont amee.
Rameo, defor thar nor fothame an which is no part of thee
By namy;
Or, if thou not pamy d O, isher willl are is n.

IET:
I amis amy amy ene
Thou at thy word would, sme she not wo bould,
ner Retake d wit;
Call non bet be ff.

ROMEO:
Ar, athat kis nor he.
Ar m

Part 5 — Short Analysis

Greedy decoding always takes the single most likely next character. Because the model was trained (and lightly overfit — final training loss ≈ 0.37) on a very short, highly repetitive scene, greedy decoding mostly reproduces the memorized training text almost verbatim, with only occasional small character-level slips (e.g. doubled letters like “itt writtten”). After running past the length of a few memorized lines it also loops back and repeats an earlier line (“By a name / I kno[w] not how to...”) almost verbatim — a small-scale example of the classic **repetition-loop** failure mode of greedy decoding, visible even in this tiny, overfit setting. This makes greedy the **most coherent** output observed here, but the coherence is really memorization rather than generalization — on a larger, less-repeated corpus greedy decoding tends to fall into these repetitive loops much more severely.

Temperature sampling shows the expected gradient. At $T=0.5$ the output stays close to the greedy/memorized text and reproduces long, grammatical, recognizable stretches of the source scene — still fairly coherent. At $T=1.0$, sampling draws from the full, unmodified distribution and errors accumulate quickly (dropped, duplicated, or substituted letters, merged words), giving output that is noticeably less coherent than $T=0.5$ but visibly more varied. At $T=1.5$, the distribution is flattened the most and the output degenerates into largely unrecognizable character strings, only occasionally interrupted by short bursts of real words or names (“Romeo”, “Juliet”) — the **most diverse** setting, but also the least coherent, since it samples too much probability mass from unlikely characters.

Top-k sampling behaves similarly to temperature sampling but with a hard cutoff on the candidate set rather than a smooth reshaping of probabilities. $k=5$ restricts each step to the 5 most likely characters, which kept generation close to the coherent, low-temperature regime — closely tracking real lines from the scene. $k=20$ allows a much larger share of the vocabulary through, and produced noticeably more spelling breakdowns and fewer complete words than $k=5$ — a clearly more diverse (and less coherent) result.

Conclusion: greedy decoding (and low-temperature / small-k sampling) produces the most coherent text, at the cost of largely reciting the training data and occasionally looping back on itself; high-temperature sampling ($T=1.5$) produces the most diverse text, at the cost of coherence. Top-k sampling with a moderate k is the useful middle ground — it restricts sampling to plausible characters (avoiding the worst nonsense of unrestricted high-temperature sampling) while still injecting more variety than greedy decoding or very low temperatures, making it the strategy most likely to generalize well on a larger, non-repetitive training corpus.